

06__walkforward

June 30, 2026

1 06 — Walk-forward, point-in-time backtest (the real test)

This is the rigorous version of notebook 05. It removes the two biases that made the in-sample result untrustworthy:

- **No look-ahead on skill.** At each rebalance date T, the roster is ranked using *only bets that had already resolved before T*. A trader earns a spot on past performance, exactly like real life.
- **Realistic entry price.** We buy at the market price that actually existed at T (from CLOB price-history), not the sharp's original entry. So we pay what a follower would really pay.

The signal being tested = co-holding consensus: at date T, how many *point-in-time* roster sharps were *simultaneously holding* the same market-side (entered, not yet resolved). We then buy at T's price, hold to resolution, and measure the result out-of-sample, broken down by backer count.

1.0.1 What this STILL can't fix (be honest about it)

1. **Candidate-pool survivorship.** Our candidate universe is *today's* leaderboard, so we only ever consider traders who ended up successful. Polymarket exposes no *historical* leaderboard, so this residual bias can't be removed from here. It inflates results somewhat — but far less than notebook 05, because skill ranking and outcomes are now point-in-time.
2. **Market-first holder density (#2) is live-only.** The top-holders endpoint is a snapshot with no history, so “market dense with sharps” cannot be reconstructed for past dates. It belongs in the forward engine (notebook 04), not this backtest.

So: if edge **survives here and grows with backers**, that's real evidence. If it collapses, the strategy was mostly survivorship. Either way you learn the truth cheaply.

```
[1]: import importlib, pmc
importlib.reload(pmc)
from pmc import CFG, get_leaderboard, get_closed_positions, price_at
import pandas as pd, numpy as np, time
from datetime import datetime, timezone

NOW = int(time.time())
print("Walk-forward config:",
      f"\n  candidates={CFG.WF_CANDIDATES}  lookback={CFG.WF_LOOKBACK_MONTHS}mo",
      "\n  rebalance={CFG.WF_REBALANCE_MONTHS}mo  roster={CFG.WF_ROSTER_SIZE}"
      f"\n  realistic price (price-history) = {CFG.WF_USE_PRICE_HISTORY}")
```

Walk-forward config:

```
candidates=150 lookback=12mo rebalance=3mo roster=50
realistic price (price-history) = True
```

1.1 1. Candidate pool + full resolved history

We pull a wide candidate pool (current leaderboard) and every candidate's resolved bets, keeping the timestamps we need to reconstruct *who held what, when*.

```
[2]: cands = {}
for w in ("ALL", "MONTH"):
    for r in get_leaderboard(window=w, limit=CFG.WF_CANDIDATES):
        cands.setdefault(r["wallet"], r)
print(f"{len(cands)} candidate wallets")

rows = []
for i, wallet in enumerate(cands):
    for p in get_closed_positions(wallet, max_positions=600):
        rows.append({
            "wallet": wallet,
            "asset": p.get("asset"),
            "conditionId": p.get("conditionId"),
            "outcome": p.get("outcome"),
            "entry": float(p.get("avgPrice") or 0),
            "entry_ts": int(p.get("timestamp") or 0),
            "endDate": p.get("endDate"),
            "won": 1 if float(p.get("realizedPnl") or 0) > 0 else 0,
        })
    if (i + 1) % 25 == 0:
        print(f"  pulled {i+1}/{len(cands)}")

h = pd.DataFrame(rows)
# resolution time from endDate; drop rows we can't place in time
h["res_ts"] = (pd.to_datetime(h["endDate"], errors="coerce", utc=True).
    ↪ astype("int64") // 10**9)
h = h[(h["entry_ts"] > 0) & (h["res_ts"] > 0) & (h["res_ts"] > h["entry_ts"]) &
    ↪ (h["entry"] > 0)]
print(f"{len(h)} usable resolved bets across {h['conditionId'].nunique()}
    ↪ markets, "
    f"{h['wallet'].nunique()} wallets")
```

249 candidate wallets

```
pulled 25/249
pulled 50/249
pulled 75/249
pulled 100/249
pulled 125/249
pulled 150/249
```

```

pulled 175/249
pulled 200/249
pulled 225/249
20779 usable resolved bets across 10414 markets, 209 wallets

```

1.2 2. Point-in-time roster + co-holding signal functions

`pit_roster(T)` ranks wallets using only bets resolved before `T`. `holdings_at(T, roster)` finds market-sides open at `T` held by roster wallets, with their backer count.

```

[3]: def pit_roster(T):
    past = h[h["res_ts"] < T]                                # only already-resolved-by-T
    ↪bets
    if past.empty:
        return set()
    g = past.groupby("wallet").agg(n=("won", "size"), wr=("won", "mean"))
    g = g[(g["n"] >= CFG.WF_MIN_TRAILING_TRADES) & (g["wr"] >= CFG.
    ↪WF_MIN_TRAILING_WINRATE)]
    g = g.sort_values("wr", ascending=False).head(CFG.WF_ROSTER_SIZE)
    return set(g.index)

def holdings_at(T, roster):
    open_now = h[(h["wallet"].isin(roster)) & (h["entry_ts"] <= T) &
    ↪(h["res_ts"] > T)]
    out = []
    for (cid, outcome, asset), g in open_now.groupby(["conditionId", "outcome",
    ↪"asset"]):
        out.append({
            "conditionId": cid, "outcome": outcome, "asset": asset,
            "backers": g["wallet"].nunique(),
            "won": int(g["won"].iloc[0]),                      # resolution is the
            ↪same for all
            "sharp_entry": round(g["entry"].median(), 4),     # fallback price if
            ↪no history
            "res_ts": int(g["res_ts"].iloc[0]),
        })
    return pd.DataFrame(out)

```

1.3 3. Walk forward

Step through rebalance dates. At each, build the PIT roster, find co-held market-sides, price them at `T` (realistically), and record one bet per market-side (no re-entering a market we already took).

```

[4]: end_date = NOW - 90 * 86400                             # leave 90d for
    ↪resolution
    start_date = end_date - CFG.WF_LOOKBACK_MONTHS * 30 * 86400
    step = CFG.WF_REBALANCE_MONTHS * 30 * 86400
    rebalance_dates = list(range(start_date, end_date + 1, step))

```

```

print(f"{len(rebalance_dates)} rebalance dates from "
      f"{datetime.datetime.utcfromtimestamp(start_date):%Y-%m-%d} to {datetime.
↳utcfromtimestamp(end_date):%Y-%m-%d}")

trades, entered = [], set()
for T in rebalance_dates:
    roster = pit_roster(T)
    if not roster:
        continue
    hold = holdings_at(T, roster)
    if hold.empty:
        continue
    for _, r in hold.iterrows():
        key = (r["conditionId"], r["outcome"])
        if key in entered:
            continue
        # realistic price at T (fallback to sharps' entry if history missing/
↳disabled)
        buy = None
        if CFG.WF_USE_PRICE_HISTORY:
            buy = price_at(r["asset"], T)
        if buy is None:
            buy = r["sharp_entry"]
        buy = min(float(buy) + CFG.SLIPPAGE_TOLERANCE, 0.99)
        if not (CFG.PRICE_FLOOR <= buy <= CFG.PRICE_CEILING):
            continue
        entered.add(key)
        trades.append({
            "T": datetime.datetime.utcfromtimestamp(T).strftime("%Y-%m-%d"),
            "title_id": r["conditionId"][:10], "outcome": r["outcome"],
            "backers": int(r["backers"]), "buy": round(buy, 3),
            "won": int(r["won"]),
            "ret": round(((1 - buy) / buy) if r["won"] else -1.0, 3),
        })
    print(f"  {datetime.datetime.utcfromtimestamp(T):%Y-%m-%d}: roster={len(roster)} "
          f"holdings={len(hold)} cumulative_trades={len(trades)}")

tr = pd.DataFrame(trades)
print(f"\n{n{len(tr)}} out-of-sample trades simulated")
tr.head(20)

```

5 rebalance dates from 2025-04-06 to 2026-04-01

/var/folders/qh/ghmfqnls4rb9thf8zw40g8w80000gn/T/ipykernel_12179/334691002.py:6:
 DeprecationWarning: datetime.datetime.utcfromtimestamp() is deprecated and
 scheduled for removal in a future version. Use timezone-aware objects to
 represent datetimes in UTC: datetime.datetime.fromtimestamp(timestamp,
 datetime.UTC).

```

f"{datetime.datetime.utcfromtimestamp(start_date):%Y-%m-%d} to
{datetime.datetime.utcfromtimestamp(end_date):%Y-%m-%d}")
/var/folders/qh/ghmfqnls4rb9thf8zw40g8w80000gn/T/ipykernel_12179/334691002.py:31
: DeprecationWarning: datetime.datetime.utcfromtimestamp() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects to
represent datetimes in UTC: datetime.datetime.fromtimestamp(timestamp,
datetime.UTC).
"T": datetime.datetime.utcfromtimestamp(T).strftime("%Y-%m-%d"),
/var/folders/qh/ghmfqnls4rb9thf8zw40g8w80000gn/T/ipykernel_12179/334691002.py:37
: DeprecationWarning: datetime.datetime.utcfromtimestamp() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects to
represent datetimes in UTC: datetime.datetime.fromtimestamp(timestamp,
datetime.UTC).
print(f" {datetime.datetime.utcfromtimestamp(T):%Y-%m-%d}: roster={len(roster)} ")

2025-04-06: roster=5 holdings=116 cumulative_trades=56

/var/folders/qh/ghmfqnls4rb9thf8zw40g8w80000gn/T/ipykernel_12179/334691002.py:31
: DeprecationWarning: datetime.datetime.utcfromtimestamp() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects to
represent datetimes in UTC: datetime.datetime.fromtimestamp(timestamp,
datetime.UTC).
"T": datetime.datetime.utcfromtimestamp(T).strftime("%Y-%m-%d"),
/var/folders/qh/ghmfqnls4rb9thf8zw40g8w80000gn/T/ipykernel_12179/334691002.py:37
: DeprecationWarning: datetime.datetime.utcfromtimestamp() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects to
represent datetimes in UTC: datetime.datetime.fromtimestamp(timestamp,
datetime.UTC).
print(f" {datetime.datetime.utcfromtimestamp(T):%Y-%m-%d}: roster={len(roster)} ")

2025-07-05: roster=6 holdings=57 cumulative_trades=85

/var/folders/qh/ghmfqnls4rb9thf8zw40g8w80000gn/T/ipykernel_12179/334691002.py:31
: DeprecationWarning: datetime.datetime.utcfromtimestamp() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects to
represent datetimes in UTC: datetime.datetime.fromtimestamp(timestamp,
datetime.UTC).
"T": datetime.datetime.utcfromtimestamp(T).strftime("%Y-%m-%d"),
/var/folders/qh/ghmfqnls4rb9thf8zw40g8w80000gn/T/ipykernel_12179/334691002.py:37
: DeprecationWarning: datetime.datetime.utcfromtimestamp() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects to
represent datetimes in UTC: datetime.datetime.fromtimestamp(timestamp,
datetime.UTC).
print(f" {datetime.datetime.utcfromtimestamp(T):%Y-%m-%d}: roster={len(roster)} ")

2025-10-03: roster=12 holdings=147 cumulative_trades=136

/var/folders/qh/ghmfqnls4rb9thf8zw40g8w80000gn/T/ipykernel_12179/334691002.py:31
: DeprecationWarning: datetime.datetime.utcfromtimestamp() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects to
represent datetimes in UTC: datetime.datetime.fromtimestamp(timestamp,

```

```

datetime.UTC).
    "T": datetime.utcnow().strftime("%Y-%m-%d"),
/var/folders/qh/ghmfqnls4rb9thf8zw40g8w80000gn/T/ipykernel_12179/334691002.py:37
: DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects to
represent datetimes in UTC: datetime.datetime.fromtimestamp(timestamp,
datetime.UTC).
    print(f" {datetime.utcnow().strftime("%Y-%m-%d")}: roster={len(roster)} "

    2026-01-01: roster=22 holdings=116 cumulative_trades=184

/var/folders/qh/ghmfqnls4rb9thf8zw40g8w80000gn/T/ipykernel_12179/334691002.py:31
: DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects to
represent datetimes in UTC: datetime.datetime.fromtimestamp(timestamp,
datetime.UTC).
    "T": datetime.utcnow().strftime("%Y-%m-%d"),

    2026-04-01: roster=28 holdings=327 cumulative_trades=336

336 out-of-sample trades simulated

/var/folders/qh/ghmfqnls4rb9thf8zw40g8w80000gn/T/ipykernel_12179/334691002.py:37
: DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects to
represent datetimes in UTC: datetime.datetime.fromtimestamp(timestamp,
datetime.UTC).
    print(f" {datetime.utcnow().strftime("%Y-%m-%d")}: roster={len(roster)} "

```

```

[4]:
      T      title_id outcome  backers   buy   won   ret
0  2025-04-06  0x05a2bace    Yes        1  0.117    1  7.511
1  2025-04-06  0x0d3ca85d    Yes        1  0.877    1  0.141
2  2025-04-06  0x12852f3c    No         1  0.110    1  8.091
3  2025-04-06  0x12852f3c    Yes        1  0.771    1  0.296
4  2025-04-06  0x14618aaf    Yes        1  0.690    1  0.450
5  2025-04-06  0x19f0ff38    Yes        1  0.159    0 -1.000
6  2025-04-06  0x1fb50f65    Yes        1  0.207    0 -1.000
7  2025-04-06  0x244c75b2    Yes        1  0.335    0 -1.000
8  2025-04-06  0x24de94a9    No         1  0.802    1  0.246
9  2025-04-06  0x2a86b397    No         1  0.525    0 -1.000
10 2025-04-06  0x2a86b397    Yes        1  0.890    0 -1.000
11 2025-04-06  0x2cb1a78f    No         1  0.745    0 -1.000
12 2025-04-06  0x2cb1a78f    Yes        1  0.434    0 -1.000
13 2025-04-06  0x330f4138    Yes        1  0.382    0 -1.000
14 2025-04-06  0x3b40221a    No         1  0.719    0 -1.000
15 2025-04-06  0x4a14b29f    No         1  0.660    0 -1.000
16 2025-04-06  0x51637548    Yes        1  0.810    1  0.234
17 2025-04-06  0x52115568    Yes        1  0.134    0 -1.000
18 2025-04-06  0x570a1927    No         2  0.361    0 -1.000

```

19 2025-04-06 0x586e7ee8 Yes 1 0.275 0 -1.000

1.4 4. Out-of-sample results by backer count

Same metrics as notebook 05 — but now point-in-time and priced realistically. **edge** (hit_rate — avg buy price) and **roi_per_bet** are the verdict. Compare against the in-sample numbers from notebook 05: honest results should be **lower**.

```
[5]: def summarize(df, n):
    s = df[df["backers"] >= n]
    if s.empty:
        return None
    return {
        "min_backers": n, "n_bets": len(s),
        "hit_rate": round(s["won"].mean(), 3),
        "avg_price": round(s["buy"].mean(), 3),
        "edge": round(s["won"].mean() - s["buy"].mean(), 3),
        "roi_per_bet": round(s["ret"].mean(), 3),
    }

if len(tr):
    res = pd.DataFrame([r for r in (summarize(tr, n) for n in [1, 2, 3]) if r])
    print("WALK-FORWARD out-of-sample summary:")
    display(res)
else:
    print("No out-of-sample trades were generated. Try widening_
↳WF_LOOKBACK_MONTHS, "
        "lowering WF_MIN_TRAILING_TRADES, or raising WF_CANDIDATES in pmc.py.
↳")
```

WALK-FORWARD out-of-sample summary:

	min_backers	n_bets	hit_rate	avg_price	edge	roi_per_bet
0	1	336	0.449	0.474	-0.025	0.284
1	2	48	0.417	0.531	-0.114	-0.119
2	3	14	0.500	0.501	-0.001	0.215

1.5 How to read this — the decision

- **edge > 0 and rising with backers, on a healthy n_bets** → the strategy has real, tradeable edge that survives point-in-time testing. Next: widen the roster / add theme clustering (#3) to raise frequency, then **paper-trade** before any real money.
- **edge 0 / negative, or only a handful of bets** → copying visible smart money is too late or too rare to beat the market. The honest move is to stop, or pivot to a genuinely faster / less-efficient angle. Far better to learn this here than live.

1.5.1 Remaining caveats (don't forget)

- Candidate-pool survivorship still nudges results upward (see top of notebook).

- **timestamp** on closed positions is treated as the entry time; if Polymarket means something slightly different, co-holding windows shift a little.
- Hold-to-resolution only; no mid-trade exits.

1.5.2 If it looks promising

The clean follow-ups, in order: (1) add **theme/event clustering** so correlated contracts count as agreement; (2) port the **market-first holder-density** signal into the live engine (04); (3) paper-trade the live engine for 4–6 weeks; (4) only then consider the gated execution build (Blueprint §11).